

Nota: 10

3 puntos  Guardado

PREGUNTA 1

El consejo deliberante del partido de Tecnociudad solicitó la construcción de un sistema distribuido que permita realizar licitaciones de forma transparente y segura. Cuando el partido necesita contratar cualquier tipo de servicios, genera un pliego con la descripción de lo solicitado y espera que distintos contratistas presenten ofertas. Durante esta fase, los costos son secretos para evitar que un oferente tenga preferencia. Una vez vencido el plazo, se publica la propuesta de cada uno, junto con el precio, y se selecciona el que cumple con las condiciones con el menor costo.

La implementación del sistema asegura que estas propiedades ocurran utilizando criptografía. La descripción a alto nivel es la siguiente:

- Se publica la licitación con un ID único asociado.
- Por cada licitación, se reciben archivos cifrado con AES-GCM de parte de los oferentes interesados. Los archivos incluyen un documento con el ID de la licitación, la descripción técnica, el CUIT de la empresa y el precio.
- Al terminar el periodo de la licitación los oferentes comparten la clave utilizada, y con ello se revisan todas las ofertas, y se adjudica la ganadora.
- Si un oferente no provee la clave, queda automáticamente descalificado.
- Para transparencia, se publican todas las ofertas recibidas ya descifradas y la adjudicada en un portal público que puede ser consultado libremente.

- a) Explicar por que ningún oferente puede alterar su propuesta sin ser detectado. (1 pto.)
- b) Que medidas adicionales serían necesarias para evitar que algún administrador del sistema ignore a un proveedor diciendo que nunca entregó la clave (1 pto.)
- c) Considerar el caso de un oferente que envía un archivo a nombre de un competidor, con un precio irrisoriamente bajo que lo perjudicaría. ¿Cómo evitaría esto usando funciones criptográficas? (1 pto.)

Para la barra de herramientas, presione ALT+F10 (PC) o ALT+FN+F10 (Mac).



- a)
Ningún oferente puede alterar la propuesta sin ser detectado ya que AES-GCM provee tanto confidencialidad como integridad. Esto implica que una vez subido el archivo, cualquier intento de modificación del archivo original cambiaría el MAC asociado al cifrado AES-GCM, lo que sería fácilmente detectable para los administradores.
- b)
Una opción fácil, puede ser que una vez finalizada la licitación, todas las claves sean subidas a un sitio público directamente en el cual todos los oferentes puedan ver las enviadas, de esta forma se transparenta la entrega de las mismas. Sin embargo, esto no provee una solución total. Otra mejora podría ser que los administradores tengan una clave privada, y una vez enviada la clave de los oferentes al finalizar la licitación, la firmen y envíen la firma al oferente. De esta forma, con la clave pública asociada, el oferente puede verificar que fue firmado, y si efectivamente esto es así, el administrador no puede negarlo, ya que la firma realizada por el administrador provee no repudio.
- c)
Esto se puede evitar fácilmente utilizando un firmado digital, con un par de claves pública y privada. La clave privada asignada al oferente, y la clave pública lógicamente conocida por todos. De esta forma, luego de realizar el encriptado de su propuesta con todos los datos puede firmarlo usando la clave privada con algún protocolo como RSA o DSS, y luego los administradores pueden validar con la clave pública asociada que corresponde al oferente y no a un imitador.

PREGUNTA 2

3 puntos  Guardado

El siguiente protocolo fue propuesto para que N nodos de un sistema puedan elegir un supervisor, de tal manera que ningún nodo pueda ganar o perder sistemáticamente manipulando el intercambio:

- 1) Cada nodo genera un valor r_i de 64 bits y envía al resto $H(r_i)$
- 2) Luego de recibir todos los valores, cada nodo publica r_i
- 3) Cada nodo calcula $r_x = r_1 \text{ xor } r_2 \text{ xor } \dots \text{ xor } r_n$

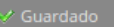
El nodo a menor distancia de r_x es el nuevo supervisor, utilizando la distancia de Hamming como métrica (se comparan bit a bit los dígitos binarios de los dos valores, y cada discrepancia suma 1 de distancia - por ejemplo 0010 y 1110 están a distancia 2). En caso de empate, se reinicia

- a) Explicar por qué el paso 1 es necesario y no se puede comenzar el intercambio en el paso 2 directamente (1 pto.)
- b) Considerar el caso de solo tres nodos. ¿Qué pasaría si el nodo 2, por ejemplo, decide abusar del sistema, y repite en el paso 1 el valor de $H(r_1)$ como si fuese el suyo? Proponer una modificación simple que evite el problema (1,5 pts.)
- c) Proponer una mejora al algoritmo para que no sea necesario repetir todo el intercambio en caso de empate y siempre se llegue a un resultado que no pueda ser manipulado. (0.5 pts.)

Para la barra de herramientas, presione ALT+F10 (PC) o ALT+FN+F10 (Mac).

- a)
El paso 1 es necesario para evitar que en el paso 2 algún nodo malicioso pueda obtener los r_i de otros nodos, luego calcular r_1 de forma que r_x quede igual a r_1 y enviarlo, obteniendo control de supervisor. Básicamente en el paso 1 se le está dando integridad al intercambio, ya que al realizar un Hash sobre r_i , ningún nodo va a poder modificar su r_i en el paso 2 para alterar el r_x a su criterio y así ganar la elección. Esto es posible ya que $H(x)$ es una función de hash resistente a colisiones, por lo que en el primer paso, nadie puede obtener la preimagen de $H(z_i)$, es decir, z_i , y de esa forma precalcular su z_i . También, como es resistente a segundas imágenes, para cualquier nodo i , le será imposible en el paso 2 cambiar su z_i por otro z_i' y lograr que $H(z_i) = H(z_i')$. De esta forma se evita que se manipule la elección modificando z_i en el segundo paso.
- b)
Si el nodo 2 decide enviar $H(z_1)$ como si fuese suyo en el paso 1, en el paso 2 podrá mandar z_1 como si fuese propio y entonces al calcular $r_x = z_1 \text{ xor } z_1 \text{ xor } z_3 = 0 \dots 0 \text{ xor } r_3 = r_3$. Entonces el nodo elegido va a ser el 3, ya que z_3 es el más similar a r_x (es igual). El nodo 2 podría, de esta forma, alterar el resultado de la elección eligiendo a su antojo tanto al nodo "1" como al nodo "2" como administrador, realizando el proceso mencionado anteriormente.
- Una forma fácil de resolverlo es que los nodos verifiquen que ningún $H(z_i)$ coincida con otro $H(z_j)$ para todo $i \neq j$, es decir, que de 2 nodos distintos no venga el mismo $H(z_i)$. De esta forma, z_i no coincidirá con ningún otro z_j . Si coincide, se resetea el proceso.
- c)
Primero, una mejora sería la mencionada en el punto b): que ningún z_i pueda coincidir con ningún otro z_j si $i \neq j$, para evitar la manipulación de la elección.
De esta forma, una segunda mejora posible en caso de empate puede ser si la distancia z_i y z_j empatan (ambos tienen la menor distancia Hamming con r_x), entonces se chequea la distancia hamming primeros $n-1$ bits de z_i y z_j con r_x , siendo n la cantidad original de bits de r_x . Si coincide de nuevo, entonces se hace con los primeros $n-2$ bits y así hasta 1. Nunca va a haber un empate total (misma distancia hamming para toda subcadena de los primeros bits desde n hasta 1) ya que eso implicaría que son iguales, violando la mejora antes mencionada.
De esta forma se logra dirimir en caso de empate y evitar manipulaciones de la elección.

PREGUNTA 3

4 puntos 

Agrolin S.A. se dedica a la construcción de soluciones tecnológicas para el campo.

Recientemente se publicó un informe que menciona múltiples vulnerabilidades en uno de sus productos y provocó muchas pérdidas de ventas a punto de concretarse. Para corregir la situación, la empresa decidió contratar un servicio de *pen testing*, realizar una evaluación mas exhaustiva y corregir el producto para empezar a recuperar el daño a su imagen.

El producto en cuestión permite hacer **un seguimiento del rendimiento de las cosechas a lo largo del tiempo** y determinar las rotaciones optimas para mejorar la productividad. El sistema está compuesto por un frontend web que es **un single page web app, donde el usuario puede registrarse, autenticarse, dar de alta lotes marcado sus limites en un mapa, actualizarlos o eliminarlos**. El sistema muestra según datos históricos de todos los clientes de la zona, el rinde esperado para la superficie en cuestión, sugiere el tipo de cultivo con mayor rentabilidad en función de **los valores de rinde y precios de mercados futuros estimados**, y permite que el **usuario ingrese los datos del tipo de cultivo y luego de la cosecha el rinde obtenido para comparar y poder mejorar en las próximas sugerencias**.

La aplicación web se ejecuta en los navegadores de cada cliente, y hace llamadas a un servidor backend **a traves de un api REST**.

Siguiendo la metodología de hipotesis de falla, establecer 4 hipotesis de vulnerabilidades que tengan alta probabilidad de existir en el escenario descrito. Para cada una, describir unicamente la hipotesis y la prueba que realizaria para confirmarla o refutarla (1 pto. por hipotesis).

Aclaraciones: No se considerarn validas hipotesis genericias. Tampoco se consideraran dos hipotesis que sean variantes del mismo tipo de falla (por ejemplo, el mismo problema en dos pantallas distintas).

Para la barra de herramientas, presione ALT+F10 (PC) o ALT+FN+F10 (Mac).

                                         	Hipotesis 1 Improper authentication -Hipotesis: El sistema cuenta con un sistema de autenticación debil o inexistente, que permite logearse facilmente a cuentas que no son propias para modificar datos o obtener datos de otros clientes. Es posible que no tenga sistema de autorizacion directamente, o use un sistema username-password que permite contraseñas debiles y realizar intentos ilimitados. -Prueba: Primero, tratar de entrar al sitio para ver si tiene algun sistema de autorizacion, en caso que no lo tenga, entrar y ver una cuenta cualquiera. En caso que tenga, una prueba facil de hacer es tratar de registrarse con credenciales simples como 'usuario'y 'contraseña'. que son facilmente adivinables. Luego, es posible realizar un ataque por diccionario en que se prueben varias contraseñas comunes para ver si podemos tener acceso.
	Hipotesis 2 SQL-Injection -Hipotesis: Al ingresar datos del campo, es posible ingresar sentencias SQL en lugares de valores normales, para provocar modificaciones en la base de datos, esto puede producirse por falta de validaciones que verifiquen los datos de entrada. -Prueba: Tras creame una cuenta, puedo introducir datos como por ejemplo: para insertar el nombre del campo, en lugar de poner solo un nombre inserto nombre; CREATE TABLE INGRESE_A_TU_BASE(int hola;)" . En caso que no haya validaciones, se creara la tabla sin realizar daños a la base de datos.
	Hipotesis 3 Denial of Service(DOS) -Hipotesis: Se menciona que el sistema puede "muestra según datos históricos de todos los clientes de la zona, el rinde esperado para la superficie en cuestión, sugiere el tipo de cultivo con mayor rentabilidad en función de los valores de rinde y precios de mercados futuros estimados". Este tipo de consulta es costosa, ya que hay que realizar varias queries y parsear los resultados, lo que genera un desbalance entre el tiempo necesario para enviar una request y lo necesario para resolver esta consulta. Por lo que es posible que si un atacante envía muchas de estas consultas en forma paralela, logre denegar el servicio por sobrecapacidad del servidor. -Prueba: NO se va a realizar este ataque para no tirar la pagina en caso de ser exitoso. A lo sumo se verificará si es posible realizar 2 consultas desde el mismo ip(con o sin autenticarse) y se informara de la situación eventualmente.
	Hipotesis 4 Improper Authorization Hipotesis: Una vez logeados, es posible que el sistema no maneje bien permisos y/o autorizaciones. De esta forma, es posible que se le permita al usuario acceder a recursos que no se le debería dar acceso, como el campo de otro cliente o recursos de administrador. Prueba: Para probar esto, tras crear una cuenta y loggearme, podría verificar la forma que tiene una url que muestra campo creado por mi. si tiene la forma: https://agronlin.app.com/cliente/12/campo/1, podría tratar de acceder al campo de otro cliente usando la url https://agronlin.app.com/cliente/1/campo/1, accediendo a recursos no deseados. No modificaria nada para no ser intrusiva la prueba, solo se muestra que se tuvo acceso, pero un atacante eventualmente podría borrar campos, etc.

PREGUNTA 1

3 puntos

Guardado

El consejo deliberante del partido de Tecnociudad solicitó la construcción de un sistema distribuido que permita realizar licitaciones de forma transparente y segura. Cuando el partido necesita contratar cualquier tipo de servicios, genera un pliego con la descripción de lo solicitado y espera que distintos contratistas presenten ofertas. Durante esta fase, **los costos son secretos** para evitar que un oferente tenga preferencia. Una vez vencido el plazo, se publica la propuesta de cada uno, junto con el precio, y se selecciona el que **cumple con las condiciones con el menor costo**.

La implementación del sistema asegura que estas propiedades ocurran utilizando criptografía. La descripción a alto nivel es la siguiente:

- Se publica la licitación con un **ID único asociado**. ^{counter mode}
- Por cada licitación, se reciben archivos cifrados con **AES-GCM** de parte de los oferentes interesados. Los archivos incluyen un documento con el ID de la licitación, la descripción técnica, el CUIT de la empresa y el precio.
- Al terminar el periodo de la licitación los oferentes **comparten la clave utilizada**, y con ello se revisan todas las ofertas, y se adjudica la ganadora.
- Si un oferente **no provee la clave**, queda automáticamente **descalificado**.
- Para transparencia, se publican todas las ofertas **recibidas ya descifradas** y la adjudicada en un portal público que puede ser consultado libremente.

- a) Explicar por qué ningún oferente puede alterar su propuesta sin ser detectado. (1 pto.)
- b) ¿Qué medidas adicionales serían necesarias para evitar que algún administrador del sistema ignore a un proveedor diciendo que nunca entregó la clave? (1 pto.)
- c) Considerar el caso de un oferente que envía un archivo a nombre de un competidor, con un precio irrisoriamente bajo que lo perjudicaría. ¿Cómo evitaría esto usando funciones criptográficas? (1 pto.)

- a) xq cada ofereta es cifrada con AES-GCM que mantiene integridad y confidencialidad, x lo tanto, cualquier cambio sera detectado.
- b) admin tenga clave privada para firma aquellos claves recibidas y enviar certificado a los oferantes asi no puede negar recepción
- c) cada ofertante tenga que firmar oferta asi despues se valida si coinciden firmas con la oferta

PREGUNTA 2

El siguiente protocolo fue propuesto para que N nodos de un sistema puedan elegir un supervisor, de tal manera que ningún nodo pueda ganar o perder sistemáticamente manipulando el intercambio:

- 1) Cada nodo genera un valor r_i de 64 bits y envía al resto $H(r_i)$
- 2) Luego de recibir todos los valores, cada nodo publica r_i
- 3) Cada nodo calcula $r_x = r_1 \text{ xor } r_2 \text{ xor } \dots \text{ xor } r_n$

El nodo a menor distancia de r_x es el nuevo supervisor, utilizando la distancia de Hamming como métrica (se comparan bit a bit los dígitos binarios de los dos valores, y cada discrepancia suma 1 de distancia - por ejemplo 0010 y 1110 están a distancia 2). En caso de empate, se reinicia

- a) Explicar por qué el paso 1 es necesario y no se puede comenzar el intercambio en el paso 2 directamente (1 pto.)
- b) Considerar el caso de solo tres nodos. ¿Qué pasaría si el nodo 2, por ejemplo, decide abusar del sistema, y repite en el paso 1 el valor de $H(r_1)$ como si fuese el suyo? Proponer una modificación simple que evite el problema (1,5 pts.)
- c) Proponer una mejora al algoritmo para que no sea necesario repetir todo el intercambio en caso de empate y siempre se llegue a un resultado que no pueda ser manipulado. (0,5 pts.)